

# CSC 250: Foundations of Computer Science I

Spring 2026

## Chapter 2: Facts about Functions

Sakhawat Hossan

Department of Computer Science  
University of North Carolina at Greensboro

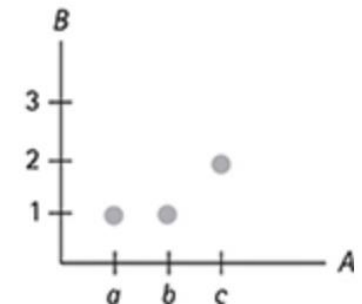
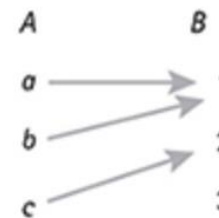
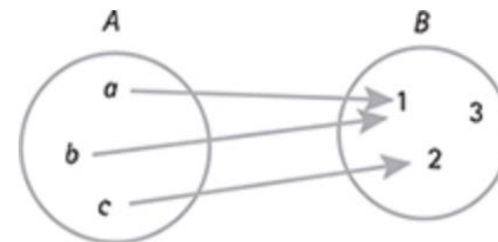
# Section 2.1 Functions: Definitions and Examples

**Function:** Suppose  $A$  and  $B$  are sets, and for each element in  $A$  we associate exactly one element in  $B$ . Such an association is called a function from  $A$  to  $B$ .

If  $f$  is a function from  $A$  to  $B$  and  $f$  associates the element  $x \in A$  with the element  $y \in B$ , then we write  $f(x) = y$  or  $y = f(x)$ .

The expression  $f(x)$  is read, “ $f$  of  $x$ ,” or “ $f$  at  $x$ ,” or “ $f$  applied to  $x$ .”

If  $f$  is a function from  $A$  to  $B$ , we denote this by writing  $f: A \rightarrow B$ .



# Function Example

*Example.* The picture shows a function  $f : A \rightarrow B$  with domain  $A = \{a, b, c, d\}$  and codomain  $B = \{1, 2, 3\}$ .

We have:  $f(a) = f(b) = 1$  and  $f(c) = f(d) = 3$ .

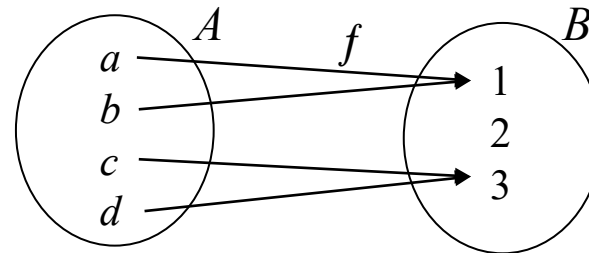
Some sample sets are:

$$\text{range}(f) = \{1, 3\},$$

$$f(\{a, b\}) = \{1\},$$

$$f^{-1}(\{2\}) = \emptyset, \text{ (inverse image )}$$

$$f^{-1}(\{1, 2, 3\}) = \{a, b, c, d\}.$$



# Floor and Ceiling

The *floor* and *ceiling* functions have type  $\mathbf{R} \rightarrow \mathbf{Z}$ , where  $\text{floor}(x)$  is the closest integer less than or equal to  $x$  and  $\text{ceiling}(x)$  is the closest integer greater than or equal to  $x$ .

e.g.,  $\text{floor}(2.6) = 2$ ,  $\text{floor}(-2.1) = -3$ ,  $\text{ceiling}(2.6) = 3$ , and  $\text{ceiling}(-2.1) = -2$ .

**Notation:**  $\lfloor x \rfloor = \text{floor}(x)$  and  $\lceil x \rceil = \text{ceiling}(x)$ .

*Prove:*  $-\lfloor x \rfloor = \lceil -x \rceil$ .

*Proof:* If  $x \in \mathbf{Z}$ , then  $-\lfloor x \rfloor = -x = \lceil -x \rceil$ . If  $x \notin \mathbf{Z}$ , there is an  $n \in \mathbf{Z}$  such that  $n < x < n + 1$ , which gives  $\lfloor x \rfloor = n$ . Multiply the inequality by  $-1$  to get  $-n > -x > -(n + 1)$ , which gives  $\lceil -x \rceil = -n$ .

So  $-\lfloor x \rfloor = -n = \lceil -x \rceil$ .

# Greatest Common Divisor (gcd)

A **divisor** of an integer  $n$  is an integer that divides  $n$  evenly

Example: Divisors of 12

$$\{1, 2, 3, 4, 6, 12\}$$

Common divisors of 12 and 18:

$$\{1, 2, 3, 6\}$$

Greatest common divisor = 6

**GCD:** If  $x$  and  $y$  are integers, not both zero, then  $\gcd(x, y)$  is the largest integer that divides  $x$  and  $y$ .

For example,  $\gcd(12, 15) = 3$ ,  $\gcd(-12, -8) = 4$ .

# Greatest Common Divisor (gcd)

## Properties of gcd

- $\gcd(a, b) = \gcd(b, a) = \gcd(a, -b)$ .
- $\gcd(a, b) = \gcd(b, a - bq)$  for any integer  $q$ .
- $\gcd(a, b) = ma + nb$  for some  $m, n \in \mathbf{Z}$ .
- If  $d \mid ab$  and  $\gcd(d, a) = 1$ , then  $d \mid b$ .

**Euclid's algorithm** for finding  $\gcd(a, b)$

Assume  $a, b \in \mathbf{N}$ , not both zero.

**while**  $b > 0$  **do**

    find  $q, r$  so that  $a = bq + r$  and  $0 \leq r < b$ ;

$a := b$ ;

$b := r$

**End while**

Output( $a$ )

Example. Find  $\gcd(189, 33)$ .

$$189 = 33 \cdot 5 + 24$$

$$33 = 24 \cdot 1 + 9$$

$$24 = 9 \cdot 2 + 6$$

$$9 = 6 \cdot 1 + 3$$

$$6 = 3 \cdot 2 + 0$$

Output(3).

# The Mod Function (Remainder)

## Definition

For integers  $a$  and  $b$ , with  $b > 0$ ,  
$$a \bmod b = r$$

Where,

$$a = bq + r \text{ and } 0 \leq r < b$$

$q$ : quotient

$r$ : remainder

## Example:

$17 \bmod 5 = 2$  (because  $17 = 5 \cdot 3 + 2$ )

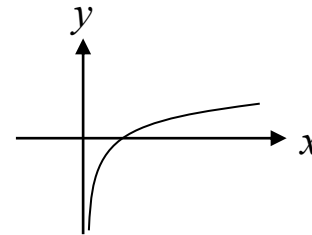
$20 \bmod 4 = 0$  (because 20 is divisible by 4)

# Log Function

## Definition

If  $x$  and  $b$  are positive real numbers and  $b > 1$ , then  $\log_b x = y$  means  $b^y = x$ . The graph of the log function is pictured at the right.

$$\log_b x = y \iff b^y = x$$



## Example:

**Base 2:**  $\log_2 n \rightarrow \log_2 8 = 3$  (because  $2^3 = 8$ )

**Base 10:**  $\log_{10} n \rightarrow \log_{10} 1000 = 3$  (because  $10^3 = 1000$ )

# Encryption

Encryption is the practice of scrambling information in a way that only someone with a corresponding key can unscramble and read it. Encryption is a two-way function. When you encrypt something, you're doing so with the intention of decrypting it later.

To encrypt data, you use something called a cipher, which is an algorithm – a series of well-defined steps that can be followed procedurally – to encrypt and decrypt information.

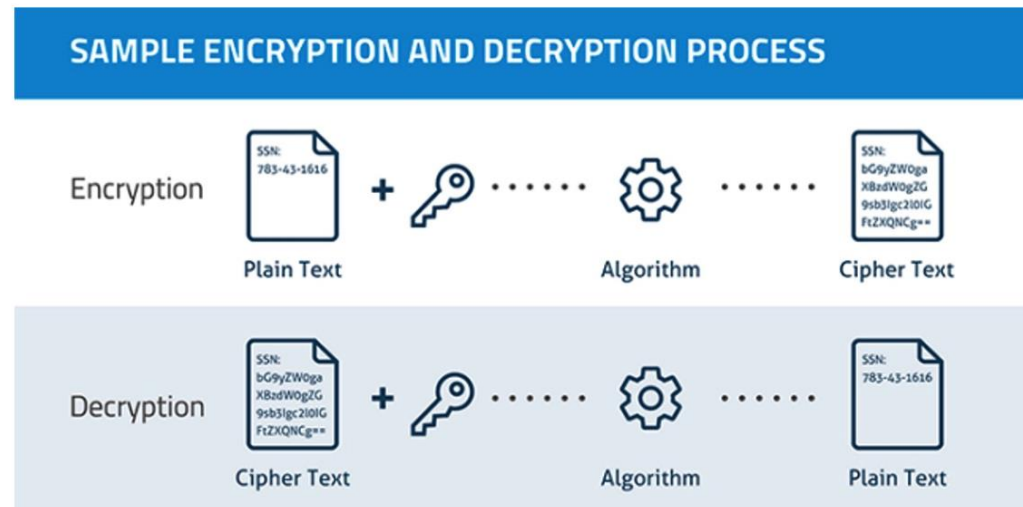


Image credit - <https://mediawiki.middlebury.edu/>

# Caesar Cipher

In cryptography, a Caesar cipher, also known as Caesar's cipher, the shift cipher, Caesar's code or Caesar shift, is one of the simplest and most widely known encryption techniques. It is a type of substitution cipher in which each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet.

For example, with a left shift of 3, D would be replaced by A, E would become B, and so on. The method is named after Julius Caesar, who used it in his private correspondence.

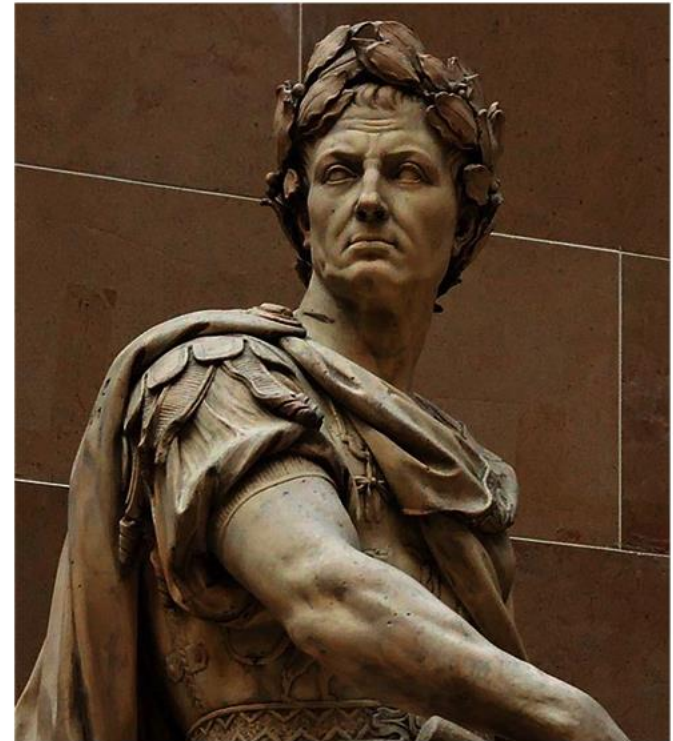


Image credit - <https://historylearning.com/>

# Caesar Cipher

The transformation can be represented by aligning two alphabets; the cipher alphabet is the plain alphabet rotated left or right by some number of positions. For instance, here is a Caesar cipher using a left rotation of three places, equivalent to a right shift of 23 (the shift parameter is used as the [key](#)):

|               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| <b>Plain</b>  | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |
| <b>Cipher</b> | X | Y | Z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W |

When encrypting, a person looks up each letter of the message in the "plain" line and writes down the corresponding letter in the "cipher" line.

Plaintext: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG  
Ciphertext: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD

Deciphering is done in reverse, with a right shift of 23

# Hill Cipher

Hill cipher is a polygraphic substitution cipher based on linear algebra. Each letter is represented by a number modulo 26. Often the simple scheme A = 0, B = 1, ..., Z = 25 is used, but this is not an essential feature of the cipher. To encrypt a message, each block of  $n$  letters (considered as an  $n$ -component vector) is multiplied by an invertible  $n \times n$  matrix, against modulus 26. To decrypt the message, each block is multiplied by the inverse of the matrix used for encryption.

The matrix used for encryption is the cipher key, and it should be chosen randomly from the set of invertible  $n \times n$  matrices (modulo 26).

- **Encryption :**

$$\text{Cipher Text} = (\text{Plain Text} \times \text{Key}) \text{ Mod } 26$$

- **Decryption:**

$$\text{Plain Text} = (\text{Cipher Text} \times \text{Key}^{-1}) \text{ Mod } 26$$

- **Message:** ATTACK IS TONIGHT

- **Key** = 
$$\begin{bmatrix} 3 & 10 & 20 \\ 20 & 9 & 17 \\ 9 & 4 & 17 \end{bmatrix}$$

- **Message:** ATTACK IS TONIGHT

- **Cipher Text = (Plain Text x Key) Mod 26**

$$= \begin{bmatrix} 0 & 19 & 19 \\ 0 & 2 & 10 \\ 8 & 18 & 19 \\ 14 & 13 & 8 \\ 6 & 7 & 19 \end{bmatrix} \times \begin{bmatrix} 3 & 10 & 20 \\ 20 & 9 & 17 \\ 9 & 4 & 17 \end{bmatrix} \text{ Mod } 26$$

# Hill Cipher

$$\begin{bmatrix} 0 & 19 & 19 \\ 0 & 2 & 10 \\ 8 & 18 & 19 \\ 14 & 13 & 8 \\ 6 & 7 & 19 \end{bmatrix} \times \begin{bmatrix} 3 & 10 & 20 \\ 20 & 9 & 17 \\ 9 & 4 & 17 \end{bmatrix} \text{ Mod } 26$$

$$C_{11} = 0*3 + 19*20 + 19*9 = 551 \text{ Mod } 26 = 05$$

$$C_{12} = 0*10 + 19*9 + 19*4 = 247 \text{ Mod } 26 = 13$$

$$C_{13} = 0*20 + 19*17 + 19*17 = 646 \text{ Mod } 26 = 22$$

ATT → **FNW** Similarly you can calculate other...

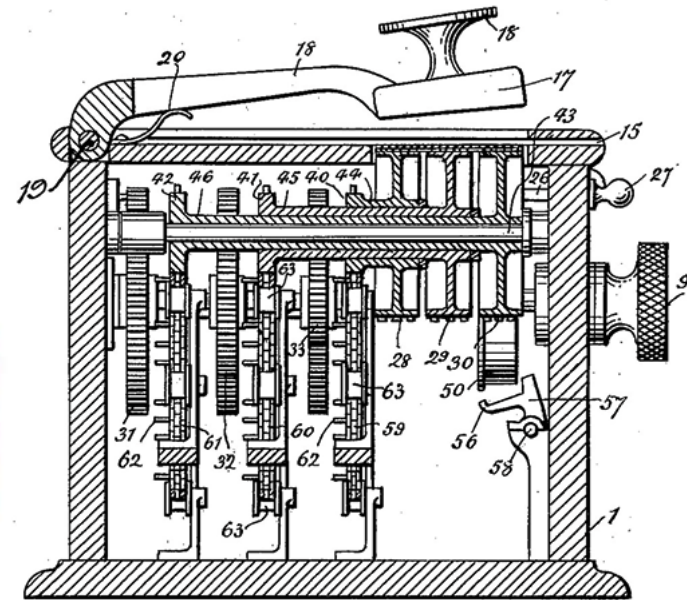


Image credit - <https://en.wikipedia.org/>

# SQL Injection

SQL is “Structured Query Language”. It is a standardized language for accessing databases. Every programming language implements SQL functionality in its own way.

Examples of SQL statements:

- `select name from employee where ssn='123456789'`
- `select name, ssn, dob from employee where ssn='123456789' and id='31042'`

# SQL Injection Example

Assume that the select statement implemented is:

- `res = select CBalance from Balances where Acct='$acct'`  
`$acct` is the variable containing the account number input

|                           |                                   |
|---------------------------|-----------------------------------|
| Enter your account number | <input type="text" value="3215"/> |
| Your balance              | <input type="text"/>              |

`res = select CBalance from Balances where Acct='3215'`

But what if the user enters something like this:

|                           |                                                    |
|---------------------------|----------------------------------------------------|
| Enter your account number | <input type="text" value="9999'%20or'%20'1'='1'"/> |
| Your balance              | <input type="text"/>                               |

```
res = select CBalance from Balances where Acct='9999' or '1'='1'
```

`res` will contain, depending on the language

- ❑ every record
- ❑ the first record
- ❑ the last record

# Hashing

Hashing is the practice of using an algorithm to map data of any size to a fixed length. This is called a hash value.

## Hashing Rules:

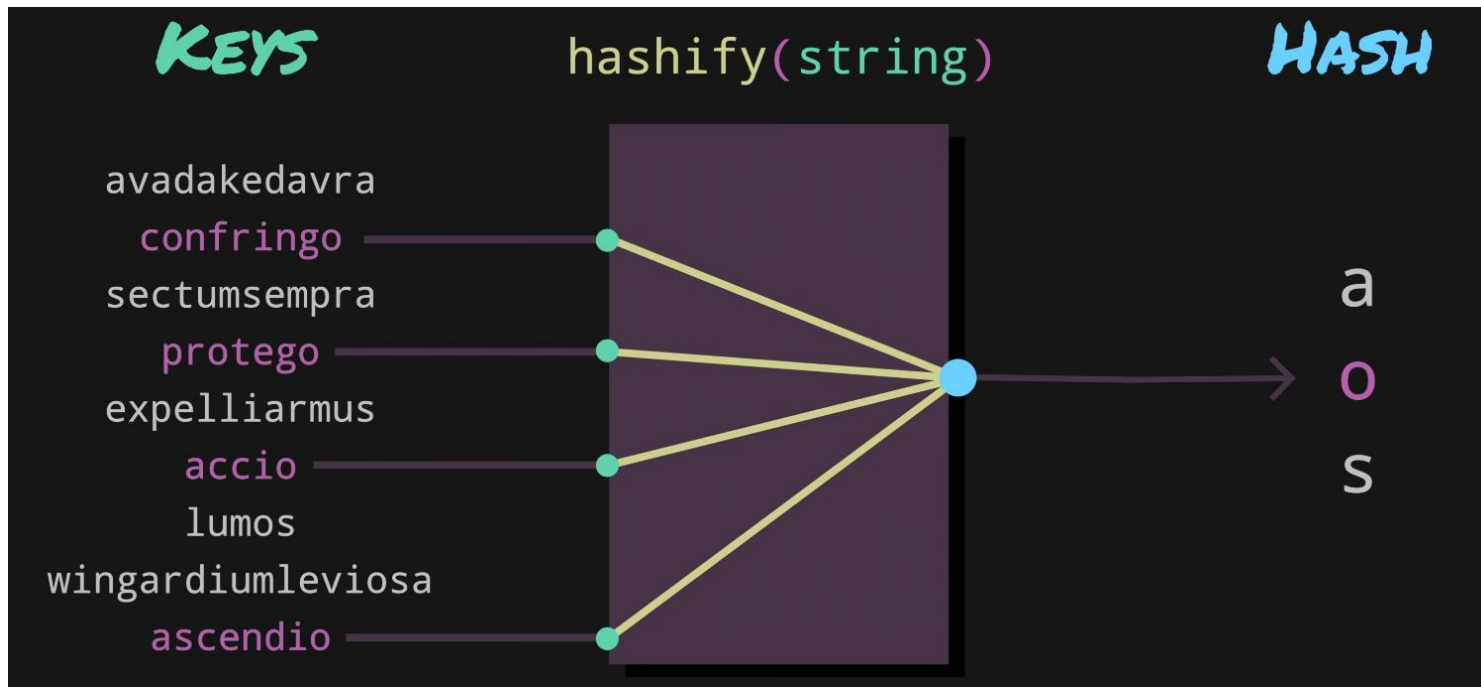
1. It must be deterministic: The same input must always have the same same output.
2. The output for all possible inputs should be of fixed length.
3. The output must be irreversible.

```
1 int add(int a, int b) {  
2     return a + b;  
3 }
```

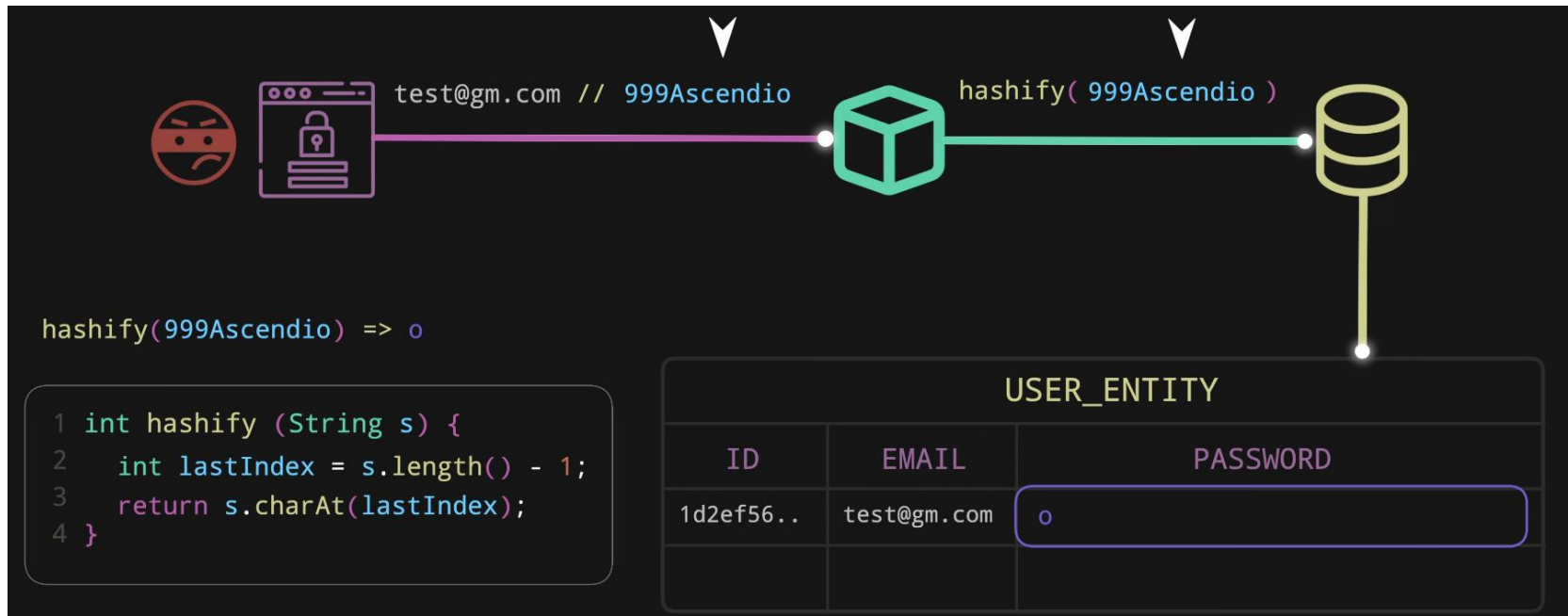
```
1 int hashify(int a, int b) {  
2     int sum = a + b;  
3     return sum % 10;  
4 }
```

# Hash Collisions

```
1 int hashify (String s) {  
2     int lastIndex = s.length() - 1;  
3     return s.charAt(lastIndex);  
4 }
```



# Hash Collisions Problem



# Popular Hashing Algorithms

| Algorithm      | Output Size | Common Use                   | Security Status       |
|----------------|-------------|------------------------------|-----------------------|
| <b>MD5</b>     | 128 bits    | Checksums, legacy systems    | ✗ Broken (collisions) |
| <b>SHA-1</b>   | 160 bits    | Legacy security, Git (older) | ✗ Broken              |
| <b>SHA-256</b> | 256 bits    | Passwords, blockchain, TLS   | ✓ Secure              |
| <b>SHA-512</b> | 512 bits    | High-security applications   | ✓ Secure              |
| <b>bcrypt</b>  | Variable    | Password hashing             | ✓ Secure              |
| <b>SHA-3</b>   | Variable    | Modern cryptography          | ✓ Secure              |

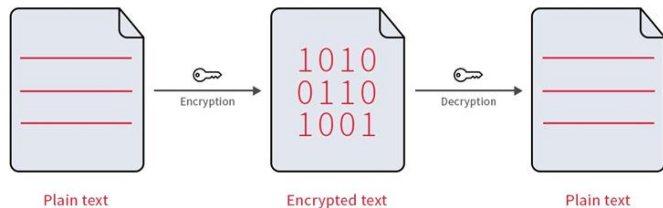
MD5 Hash Generator:

<https://www.md5hashgenerator.com/>

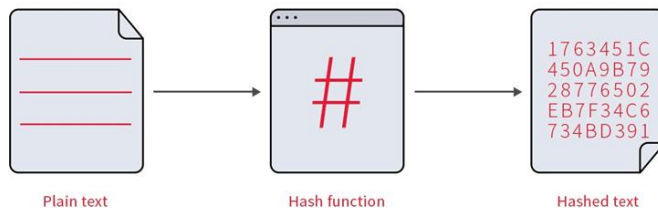
# How to Not Store Passwords

- Storing user passwords in the database as it is in plain text.
- Using an encryption method
- Using a simple hash function

## Encryption & Decryption



## Hashing Algorithm



Showing rows 0 - 11 (12 total, Query took 0.0003 sec)

```
SELECT *
FROM user
LIMIT 0, 30
```

Showing: 30 row(s) starting from record # 0  
in horizontal mode and repeat headers after 100 cells  
Sort by key: None  
+ Options

|                          | id | userid  | email                   | password | gender | country | image_url    | status |
|--------------------------|----|---------|-------------------------|----------|--------|---------|--------------|--------|
| <input type="checkbox"/> | 1  | spic    | test1@gmail.com         | spic     | male   | india   | IMG_0571.JPG | 0      |
| <input type="checkbox"/> | 17 | demo    | demo@gmail.commm        | demo     | male   | India   | IMG_0271.JPG | 0      |
| <input type="checkbox"/> | 27 | tufail  | tu@tu.com               | tufail   | male   | India   | IMG_5836.JPG | 0      |
| <input type="checkbox"/> | 26 | fuzail  | byhaqi.fuzail@yahoo.com | fuzail   | male   | India   | IMG_3150.JPG | 0      |
| <input type="checkbox"/> | 30 | manzoor | manzoor@gmail.com       | manzoor  | female | India   | user.jpg     | 0      |
| <input type="checkbox"/> | 25 | sanah   | qadrisanah@gmail.com    | sanah    | female | India   | IMG_5650.JPG | 0      |
| <input type="checkbox"/> | 33 | irfan   | irfan@gmail.com         | irfan    | male   | India   | user.jpg     | 0      |
| <input type="checkbox"/> | 52 | basit   | basit@gmail.com         | basit    | male   | Canada  | user.jpg     | 0      |
| <input type="checkbox"/> | 53 | haris   | haris@gmail.com         | haris    | male   | Canada  | user.jpg     | 0      |
| <input type="checkbox"/> | 54 | usman   | usmi@gmail.com          | usman    | male   | Canada  | user.jpg     | 0      |
| <input type="checkbox"/> | 55 | uzma    | uzma@gmail.com          | uzma     | female | Canada  | user.jpg     | 0      |
| <input type="checkbox"/> | 56 | faizan  | faizan@yahoo.com        | faizan   | male   | India   | user.jpg     | 0      |

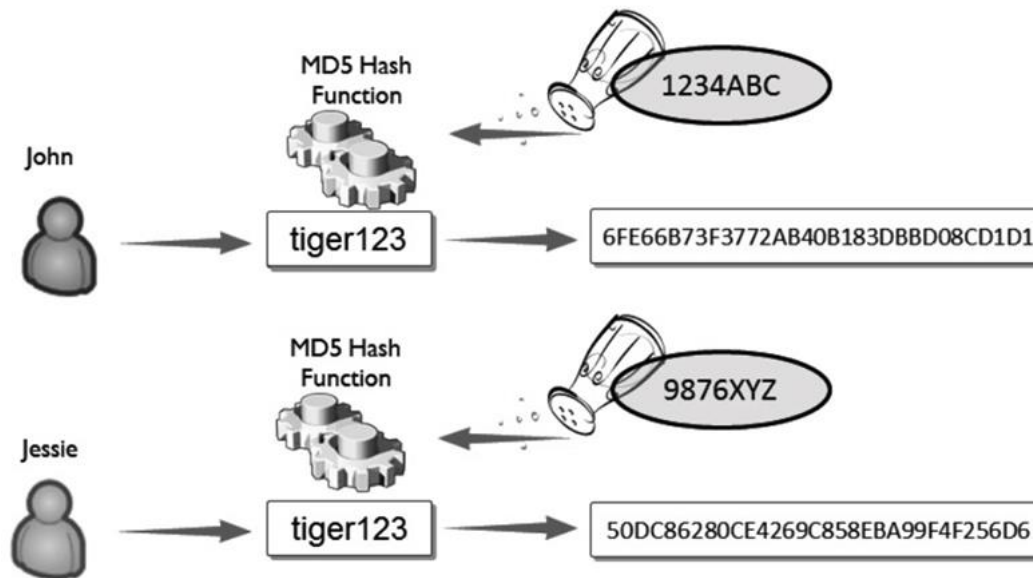
Check All / Uncheck All With selected: ☐ ☒ ☐  
Showing: 30 row(s) starting from record # 0  
in horizontal mode and repeat headers after 100 cells

# Rainbow Table Attack

A rainbow table attack is a password-cracking technique that uses a precomputed table of hash-value pairs to reverse cryptographic hashes and reveal plaintext passwords, bypassing the need for time-intensive, real-time brute force computation. It exploits weak, unsalted password hashes, allowing attackers to quickly look up hashed passwords stolen from a database to find their corresponding plaintext, making it a faster alternative to traditional cracking methods.

# Salted Secure Hash Function

A salt is added to the hashing process to force their uniqueness, increase their complexity without increasing user requirements, and to mitigate password attacks like hash tables.



# Injective (One-to-One) Function

A function

$$f: A \rightarrow B$$

is injective if different inputs always produce different outputs.

## Intuition

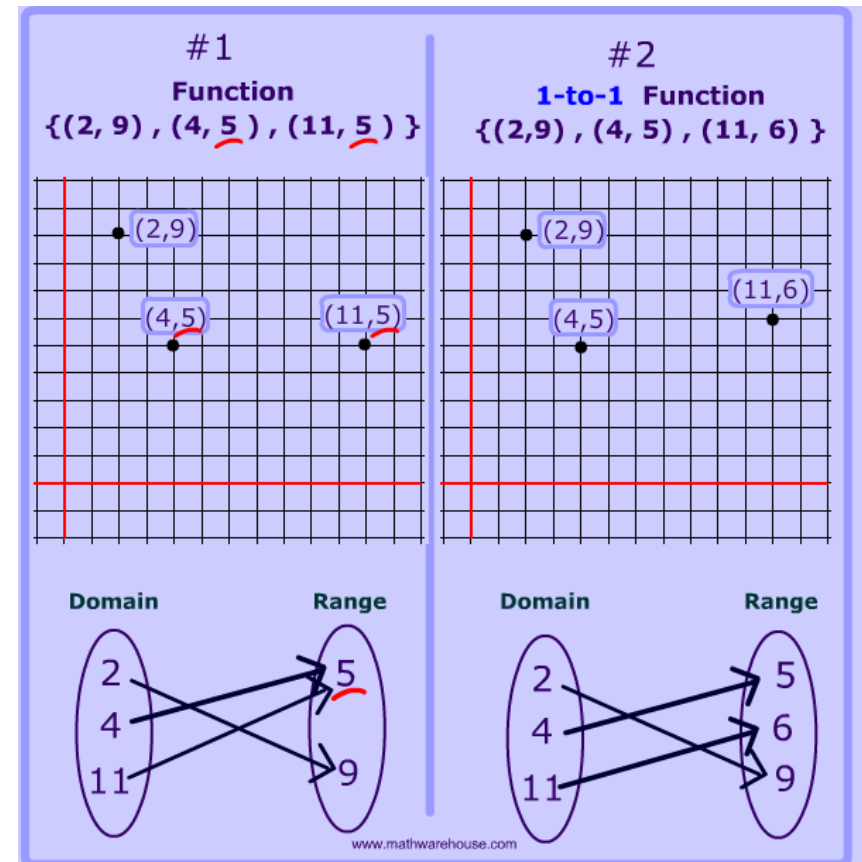
- No two elements of  $A$  map to the same element of  $B$
- Every output has at most one input
- Codomain (Range) may have unused elements

## Example:

Let  $f(x) = 2x$  from  $\mathbb{Z} \rightarrow \mathbb{Z}$

$$f(1) = 2, f(2) = 4, f(3) = 6$$

No collisions  $\rightarrow$  **injective**



# Surjective (Onto) Function

A function  $f: A \rightarrow B$  is surjective if every element of the codomain  $B$  is hit by the function.

## Intuition:

- Every element of  $B$  has at least one preimage
- Nothing in  $B$  (Codomain) is left unused

## Example:

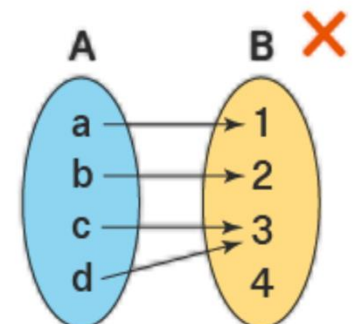
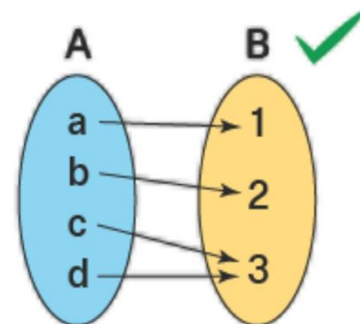
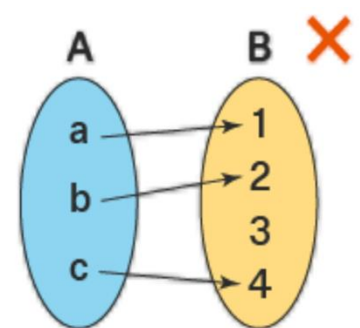
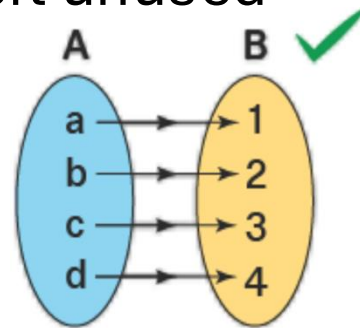
Let

$$A = \{1, 2, 3\}, B = \{a, b\}$$

Define:

$$f(1) = a, f(2) = b, f(3) = b$$

Every element of  $B$  is mapped to Surjective, even though two inputs map to  $b$



# Bijjective (One-to-One and Onto) Function

A function is bijective if it is both injective and surjective.

Formally:

Each  $b \in B$  is mapped to by exactly one element of  $A$

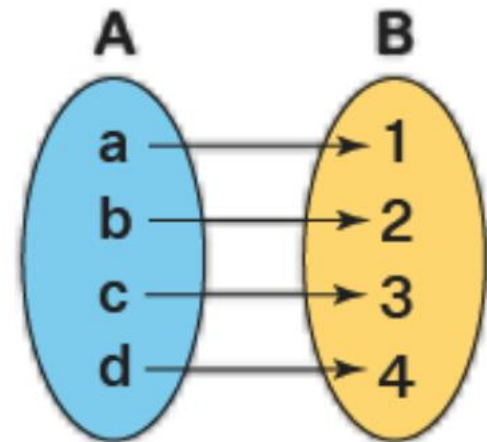
## Intuition

- Perfect pairing
- No collisions
- No leftovers

## Example

Let  $f(x) = x + 1$  from  $\mathbb{Z} \rightarrow \mathbb{Z}$

- Injective: different  $x \rightarrow$  different outputs
- Surjective: for any  $y$ , choose  $x = y - 1$
- Therefore **bijective**



# Why bijections matter

If  $f$  is bijective, then:

- The inverse function  $f^{-1}$  **exists**
- Every output can be traced back uniquely

This is why bijections are central in:

- Counting
- Cryptography
- Databases
- Discrete math proofs

# Example of a function with an inverse

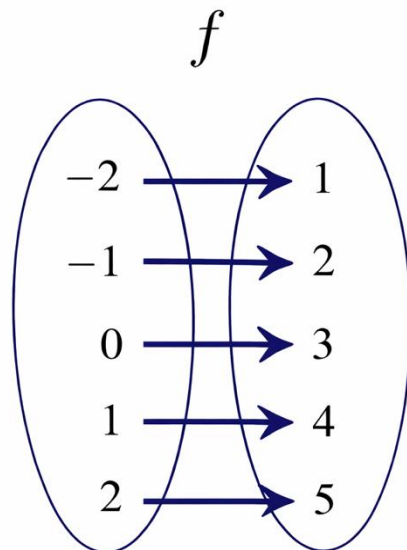
## Step 1: Define the function

Let

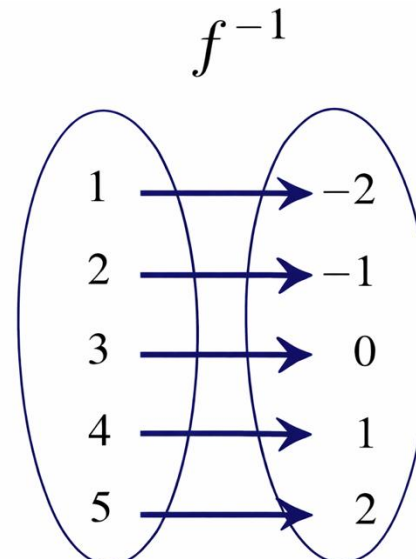
$$f: \mathbb{Z} \rightarrow \mathbb{Z}$$

be defined by

$$f(x) = x + 3$$



$$f(x) = x + 3$$



$$f^{-1}(y) = y - 3$$

# Example of a function with an inverse

## Step 2: Check that $f$ is bijective

It needs to be **Injective**

If

$$f(x) = f(y)$$

then

$$x + 3 = y + 3 \Rightarrow x = y$$

So  $f$  is injective.

It also need to be **Surjective**

Take any  $y \in \mathbb{Z}$ . Choose  $x = y - 3$ .

Then:

$$f(x) = (y - 3) + 3 = y$$

So  $f$  is surjective.

Therefore,  $f$  is **bijective**.

# Example of a function with an inverse

## Step 3: Find the inverse function

To find the inverse:

- Start with  $y = f(x)$
- Solve for  $x$

$$y = x + 3$$
$$x = y - 3$$

So, the inverse is:

$$f^{-1}(y) = y - 3$$

$$\Rightarrow f^{-1}(x) = x - 3$$

# Verify using composition

A function and its inverse must satisfy:

- Take the output and return the original input

$$f(f^{-1}(y)) = y \text{ and } f^{-1}(f(x)) = x$$

Check:

$$f(f^{-1}(y)) = f(y - 3) = (y - 3) + 3 = y$$

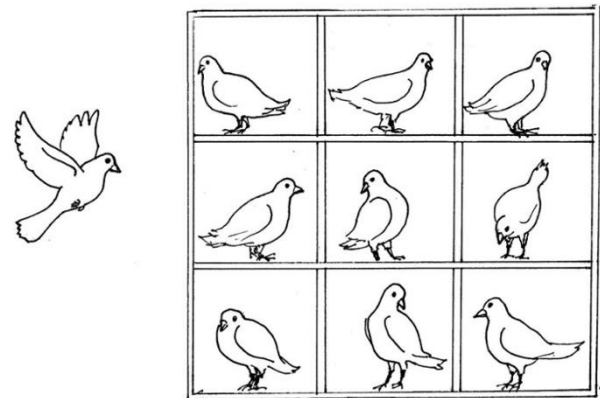
$$f^{-1}(f(x)) = f^{-1}(x + 3) = (x + 3) - 3 = x$$

# Pigeonhole Principle

If  $m$  pigeons fly into  $n$  pigeonholes where  $m > n$ , then one pigeonhole will have two or more pigeons.

We can describe the pigeonhole principle in more formal terms as follows: If  $A$  and  $B$  are finite sets with  $|A| > |B|$ , then every function from  $A$  to  $B$  maps at least two elements of  $A$  to a single element of  $B$ . This is the same as saying that no function from  $A$  to  $B$  is an injection.

**Application:** In computer science, the Pigeonhole Principle is used in hashing algorithms, where data items (keys) are assigned to hash values (buckets)



# Definitions of Numbers

- **Natural Numbers ( $\mathbb{N}$ ):** The natural numbers are the counting numbers:

$$\mathbb{N} = \{1, 2, 3, 4, \dots\}$$

- **Whole Numbers:** The whole numbers are the natural numbers together with zero:

$$\{0, 1, 2, 3, \dots\}$$

- **Integers ( $\mathbb{Z}$ ):** The integers include all positive and negative whole numbers and zero:

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$$

- **Rational Numbers ( $\mathbb{Q}$ ):** A number is rational if it can be written as a fraction: (Decimals that terminate or repeat)

$$\frac{p}{q} \text{ where } p, q \in \mathbb{Z}, q \neq 0$$

Examples:

$$\frac{1}{2}, \frac{3}{7}, -2, 2.25$$

# Definitions of Numbers

- **Real Numbers ( $\mathbb{R}$ ):** The real numbers consist of all rational and irrational numbers.

$$\mathbb{R} = \mathbb{Q} \cup \{\text{irrational numbers}\}$$

Notes

- Represent every point on the number line
  - Excludes imaginary numbers
- **Complex Numbers ( $\mathbb{C}$ ):** A **complex number** has the form:

$$a + bi \text{ where } a, b \in \mathbb{R} \text{ and } i^2 = -1$$

Examples:

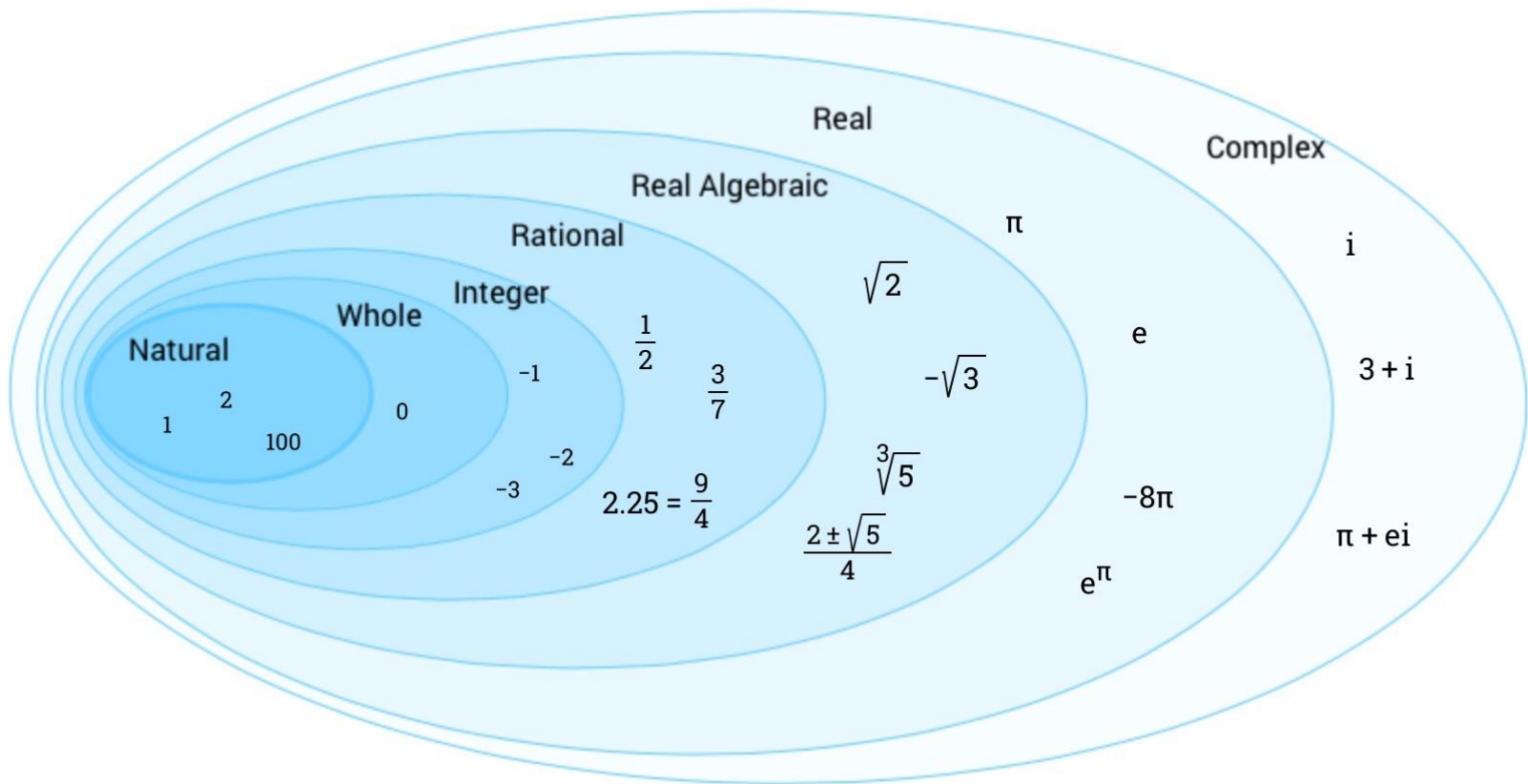
$$i, 3 + i, \pi + ei$$

Notes

- When  $b = 0$ , the number is real
- All real numbers are complex numbers

# Hierarchy of Numbers

$$\mathbb{N} \subset \text{Whole} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$$



# How Big Is a Set?

## Key idea

- In finite sets, size is easy: just count elements
- For infinite sets, “bigger” is not obvious

## Example question:

- Are there more integers than natural numbers?
- Are there more real numbers than integers?

# Countable Sets

## Definition

A set  $S$  is **countable** if:

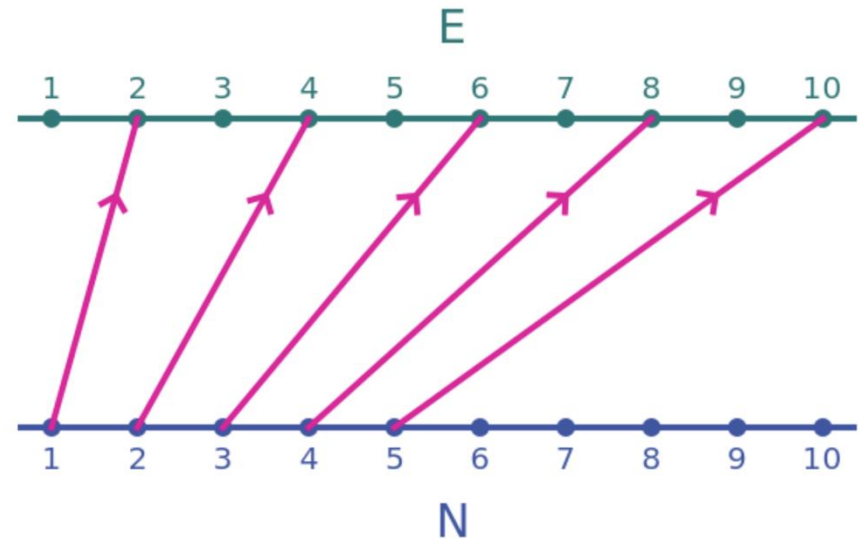
- It is **finite**, or
- Its elements can be put into **one-to-one**

**correspondence** with  $\mathbb{N} = \{1, 2, 3, \dots\}$

“Infinite but listable” = **countably infinite**

## Examples:

- $\mathbb{N} = \{1, 2, 3, \dots\}$
- $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
- $\mathbb{Q}$  (rational numbers!)



# Uncountable Sets

## Definition

A set is **uncountable** if:

- It is **infinite**, and
- **No possible listing** can include all its elements

## Key consequence

Some infinities are **strictly larger** than others

## Canonical example

The set of **real numbers between 0 and 1**, denoted:  
 $(0, 1)$

# Cantor's Diagonalization

- $|\mathbb{N}| < |\mathbb{R}|$

All real numbers in  $(0, 1)$  can be listed:

$$\begin{aligned} r_1 &= 0.d_{11}d_{12}d_{13} \dots \\ r_2 &= 0.d_{21}d_{22}d_{23} \dots \\ r_3 &= 0.d_{31}d_{32}d_{33} \dots \\ &\vdots \end{aligned}$$

Each row is a real number written in decimal form.

$$\begin{aligned} s_1 &= 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } \dots \\ s_2 &= 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } \dots \\ s_3 &= 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } \dots \\ s_4 &= 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } \dots \\ s_5 &= 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } \dots \\ s_6 &= 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } \dots \\ s_7 &= 1 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } \dots \\ s_8 &= 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } \dots \\ s_9 &= 1 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } \dots \\ s_{10} &= 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } \dots \\ s_{11} &= 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } \dots \\ &\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \ddots \end{aligned}$$

$$s = 1 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } 1 \text{ } 0 \text{ } 1 \text{ } 0 \text{ } 0 \text{ } 1 \text{ } 1 \text{ } \dots$$

# Iteration

## Selection Sort: An Iterative Sorting Algorithm

Selection sort is a simple sorting algorithm that repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it with the element at the beginning of the sorted portion. This process is repeated until the entire list is sorted.

The steps are as follows:

1. **Initialization:** The entire list is initially considered unsorted, and the sorted sublist is empty.
2. **Find Minimum:** In the first pass, the entire unsorted list is scanned to find the smallest (or largest, depending on the desired order) element.

# Iteration

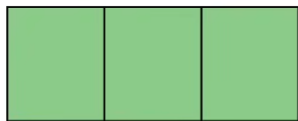
## Selection Sort: An Iterative Sorting Algorithm

- 3) **Swap:** The minimum element found is then swapped with the first element of the unsorted sublist. This element is now considered part of the sorted sublist.
- 4) **Advance Boundary:** The boundary between the sorted and unsorted parts is moved one position to the right.
- 5) **Repeat:** Steps 2-4 are repeated for the remaining unsorted portion of the list until the entire array is sorted.

For an array of  $n$  elements, this process requires  $n-1$  passes.

# Selection Sort Example

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



sorted



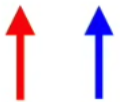
current minimum



current item

Initialization: The list is unsorted. Start from the beginning of the list

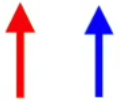
|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



# Selection Sort Example

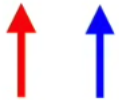
↑ current minimum  
↑ current item

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



**Step 1:** Find the minimum of the unsorted list

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



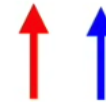
|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



# Selection Sort Example

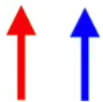
↑ current minimum  
↑ current item

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 8 | 5 | 3 | 9 | 4 | 1 |
|---|---|---|---|---|---|---|



**Swap:** Swap the minimum to the first element of the list

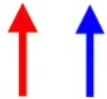
|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 8 | 5 | 3 | 9 | 4 | 2 |
|---|---|---|---|---|---|---|



# Selection Sort Example

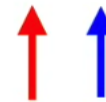
↑ current minimum  
↑ current item

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 8 | 5 | 3 | 9 | 4 | 2 |
|---|---|---|---|---|---|---|



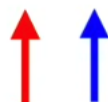
**Step 2:** Find minimum

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 8 | 5 | 3 | 9 | 4 | 2 |
|---|---|---|---|---|---|---|



**Swap:** Swap the minimum to the first element of the unsorted list

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 5 | 3 | 9 | 4 | 8 |
|---|---|---|---|---|---|---|



# Selection Sort Example

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 8 | 9 |
|---|---|---|---|---|---|---|

Repeat until the full list is sorted

## Example 2:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 6 | 3 | 7 | 9 | 8 | 2 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|